

설정 캐러셀 UI

사용자 가이드

1.0.0

By 빌리 수리아 리오에

목차 표

목차 표	1
소개	2
설치	2
데모	3
기술 세부 정보	4
UI 요소 프리팹	4
캐러셀 프로세서	5
스크립트	6
캐러셀UI 네임스페이스	6
CarouselUIElement	6
캐러셀유아이프로세서_베이스	7
CarouselUIProcessor_Template	7
CarouselUI.Demo 네임스페이스	8
CarouselProcessor_PlayerPrefs	8
Enums_PlayerPrefs	8
PreferenceTracker	8
PlayerPrefDisplayer	8
데모 씬 리스타터	9
입력 저장	10
최종 생각	11
팁	11

소개

이 에셋은 모든 단계에서 단일 옵션을 표시하는 캐러셀 UI 요소를 제작하기 위한 기반입니다. 입력 방식이 다르더라도 (포함된 데모는 Unity의 내장 입력 관리자를 사용함), 제공된 스크립트는 비슷한 동작의 UI 요소를 만드는 데 계속 사용할 수 있도록 작성되었습니다.



여러 비디오게임의 캐러셀 설정 UI 예시(위에서 아래로): *스탠리 비유*, *플아닛4*, *킹덤 컴: 날리버런스*, *디어 에스더*, *더 포레스트*.

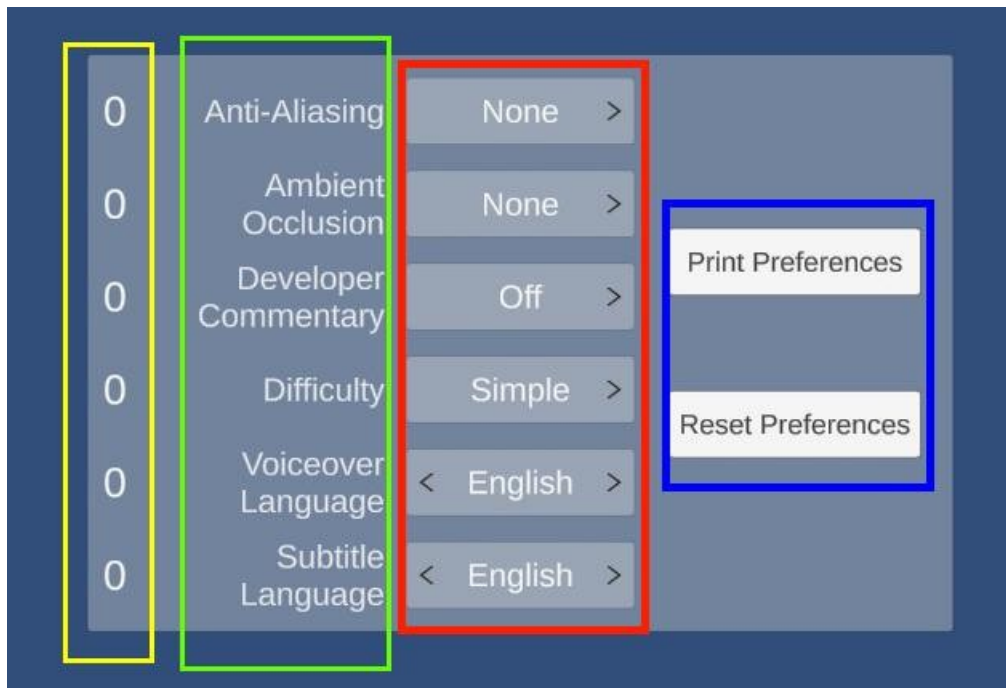
설치

패키지 관리자를 사용하거나 수동으로 패키지를 가져오기만 하면 됩니다. 새 프로젝트에서 데모 씬을 열면 텍스트 메시 프로가 TMP 에센셜 설치에 관한 창을 표시합니다. 이는 데모와 기본 프리팹이 TMP 텍스트 컴포넌트를 사용하기 때문입니다.

코드베이스로서 데모 장면과 데모 폴더(CarouselUI.Demo 네임스페이스 아래)에 있는 스크립트는 필요하지 않습니다. 물론 사용자 정의 스크립트도 포함하세요.

데모

이 에셋에는 작동 중인 UI 요소를 보여주는 데모 장면이 함께 제공됩니다. 참고용으로 스크린샷이 아래에 제공됩니다:



Red

캐러셀 UI 요소입니다. 양쪽의 버튼으로 옵션 간에 전환할 수 있습니다. 이는 에셋의 프리팹 폴더에 포함된 기본 프리팹의 변형으로 구성됩니다. 모든 상호작용은 새 설정을 플레이어 환경설정 변수로 저장합니다.

녹색

UI 요소가 제어하는 것을 나타내는 정적 텍스트입니다. 코드 기능이 없습니다.

노란색

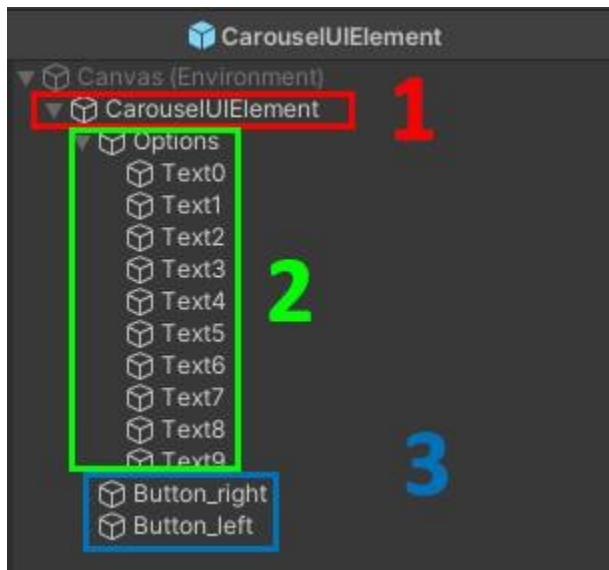
해당 캐러셀 요소에 의해 제어되는 저장된 값을 보여주는 디버그 텍스트입니다. 이러한 게임 오브젝트는 3초 후에 사라지지만 패널 맨 오른쪽에 있는 기본 설정 인쇄 버튼을 클릭하면 다시 나타납니다. 이는 동일한 행의 캐러셀 요소가 제어하는 각 플레이어 환경설정 변수를 읽는 방식으로 작동합니다.

파란색

제어 버튼. 인쇄 환경설정은 디버그 텍스트와 인쇄물을 콘솔에 토글합니다. 환경설정 재설정은 저장된 모든 환경설정을 0으로 재설정하고 씬의 UI 요소를 재설정합니다.

기술 세부 정보

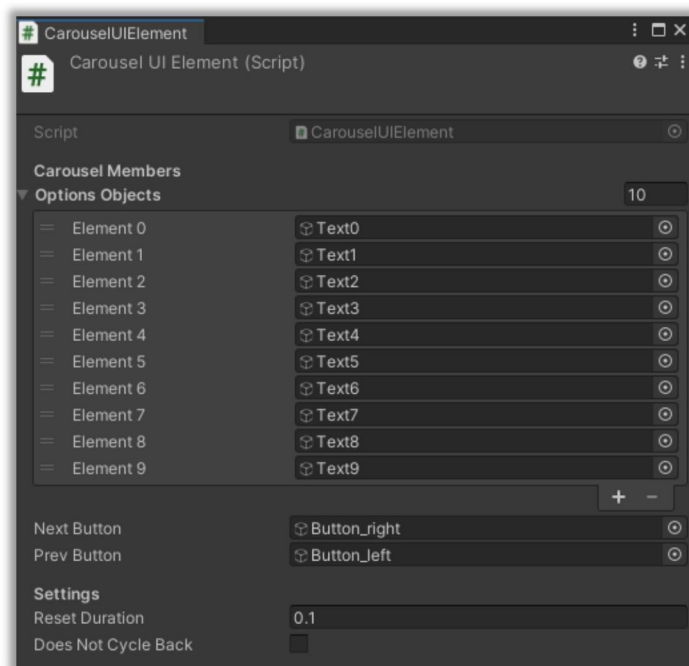
UI 요소 프리팹



CarouselUIElement 프리팹은 CarouselUI 스크립트를 구현하는 방법에 대한 기본 예제입니다. 다음으로 구성됩니다:

- 1) 루트 패널
- 2) TMPPro 텍스트 형태의 옵션 세트
- 3) 선택 항목을 앞으로 진행하기 위한 버튼(Unity의 기본 제공 버튼 UI 컴포넌트 사용)

루트 내에 포함된 CarouselUIElement 스크립트는 옵션(2)의 프리팹에 포함된 게임 오브젝트를 사용하도록 설정되어 있습니다.



또한 기본적으로 Reset Duration이 **0.1**(초)로 설정되어 있고 Does Not Cycle Back이 **False**로 설정되어 있습니다.

캐러셀 요소 스크립트를 올바르게 사용하는 한 프리팹을 직접 사용할 필요는 없습니다. 자세한 내용은 스크립트에 관한 다음 섹션에 나와 있습니다.

프리팹은 그 자체로도 완벽하게 작동하지만 어떤 효과를 내기 위해서는 캐러셀 프로세서 스크립트를 추가해야 합니다.

캐러셀 프로세서

프로세서는 캐러셀 요소가 수신하는 입력을 받아 다른 시스템으로 전달합니다. 상속할 베이스 클래스로 추상 클래스가 제공됩니다. 여기서 아이디어는 프로세서를 캐러셀에 연결하는 모든 "기능" 코드를 사용자가 원하는 추가 기능과 분리하여 유지할 수 있도록 하는 것입니다. 베이스를 상속하는 프로세서 스크립트는 데모 장면에서 사용법과 구현을 보여주기에 포함되어 있습니다. 빠른 시작 템플릿으로 사용할 수 있는 또 다른 "빈" 템플릿 스크립트도 제공됩니다.

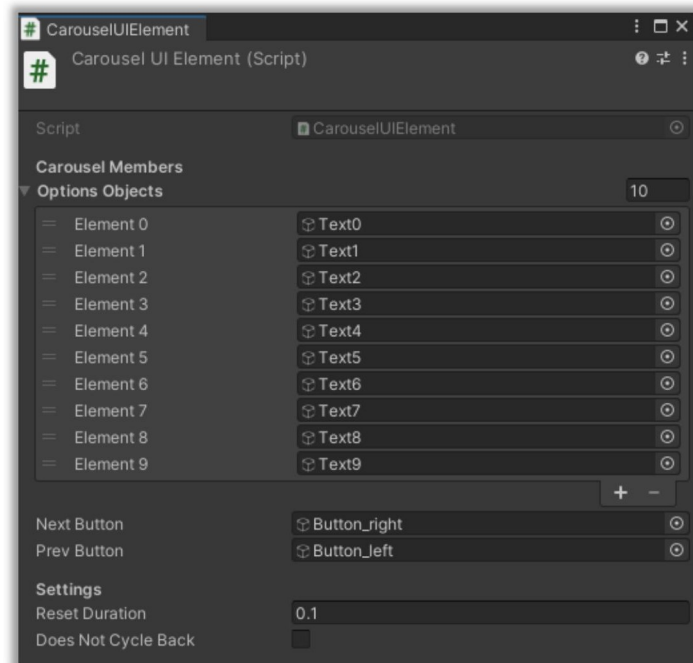
대부분의 경우 프로세서의 기본 스크립트는 추가 변경이 필요하지 않으며, 대신 템플릿 스크립트를 직접 수정하여 새 변형으로 저장할 수 있습니다.

프로세서는 CarouselUIElement 스크립트가 없는 경우 작동하지 않으며, 할당된 게임 오브젝트에서 스크립트를 찾지 못하면 오류 메시지를 생성한다는 점에 유의하세요.

스크립트

캐러셀UI 네임스페이스

CarouselUIElement



UI 어셈블리를 위한 함수형 스크립트입니다. UI 동작을 제어하고 현재 활성화된 옵션을 표시합니다. 현재 활성화된 엘리먼트 인덱스를 비공개 정수로 저장합니다.

변수

- **옵션 객체** - 캐러셀 요소에서 고려할 옵션의 배열입니다. 존재하는 요소의 수는 캐러셀에 포함될 '슬라이드'의 수에 해당합니다.
- **다음/이전 버튼** - 캐러셀과 상호 작용하는 데 사용되는 버튼에 대한 참조입니다. 여기서 참조하는 것은 단순히 게임 오브젝트에 대한 참조이므로 버튼 자체는 유니티 UI 버튼 컴포넌트일 필요는 없습니다.
- **재설정 기간** - 컴포넌트가 다시 입력을 받을 수 있을 때까지의 쿨다운 시간입니다.
- **Does Not Cycle Back** - 참이면 캐러셀이 첫 번째 또는 마지막 요소에 도달하면 각각 이전/다음 버튼을 숨깁니다.

API

- **PressNext()** - 현재 요소 인덱스를 1씩 증가시키는 데 사용됩니다.
- **PressPrevious()** - 현재 요소 인덱스를 1씩 감소시키는 데 사용됩니다.
- **UpdateIndex(int)** - 현재 요소 인덱스를 입력에 즉시 업데이트합니다.

CarouselUIProcessor_Base

캐러셀 요소에서 입력을 읽는 메서드를 포함하는 추상 클래스입니다.

변수

- AssociatedCarousel - 프로세서 인스턴스가 추적할 캐러셀 요소에 대한 캐시된 참조입니다. 사용 가능한 경우 자동으로 캐시되며, 그렇지 않으면 오류가 발생합니다.

API

- UpdateCarouselUI() - 연관된 캐러셀을 현재 설정으로 업데이트합니다. 저장된 설정의 값을 반환하고 연결된 캐러셀에 표시하려면 여기에 사용자 지정 코드를 배치해야 합니다.
- DetermineOutput() - 연결된 캐러셀에서 프로세서가 제어하는 설정을 업데이트하는 데 사용됩니다. 설정에 저장하려면 여기에 사용자 지정 코드를 배치해야 합니다.

캐러셀유아이프로세서_템플릿

```
Unity Script | 0 references
public class CarouselUIProcessor_Template : CarouselUI_Processor_Base
{
    //ADDITIONAL VARIABLES HERE TO ALLOW FOR CONTROL (REFER TO CAROUSELUIPROCESSOR_PLAYERPREFS)

    4 references
    protected override void UpdateCarouselUI()
    {
        // UPDATES THE ASSOCIATED CAROUSEL UI, FIRES ON ENABLE AS WELL AS FROM CAROUSEL ELEMENT INTERACTION
        //_storedSettingsIndex = STORE UPDATED VALUE IN THIS VARIABLE

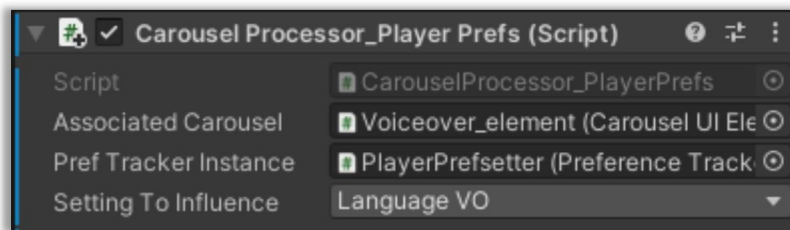
        base.UpdateCarouselUI(); //NEEDED BECAUSE THIS ALLOWS UPDATING OF THE ASSOCIATED CAROUSEL ELEMENT INDEX
    }

    2 references
    protected override void DetermineOutput(int input)
    {
        //THIS IS WHERE OUTPUT CODE GOES. I.E WHAT HAPPENS AFTER THE CAROUSEL HAS BEEN UPDATED
    }
}
```

CarouselUIProcessor_Base를 상속하는 베어본 프로세서 스크립트입니다. 적절한 기능을 위해 대체해야 하는 두 가지 주요 메서드에 대한 오버라이드를 포함합니다.

CarouselUI.Demo 네임스페이스

CarouselProcessor_PlayerPrefs



데모 장면에서 기능을 시연하는 데 사용되는 프로세서 스크립트 변형입니다. 커스텀 열거형을 사용하여 플레이어 환경설정을 통해 설정 변수를 제어합니다.

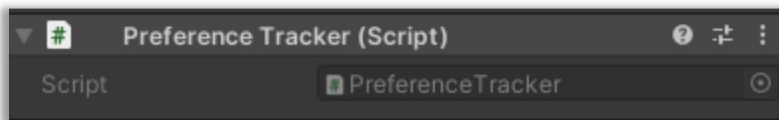
참고: 노출된 연관 캐러셀 변수는 기본 프로세서 스크립트를 상속하는 모든 스크립트의 기능이 됩니다.

Enums_PlayerPrefs

```
40 references  
public enum PreferenceEnum { A0, LanguageV0, LanguageSub, AntiAlias, DifficultyMode, DeveloperComm };
```

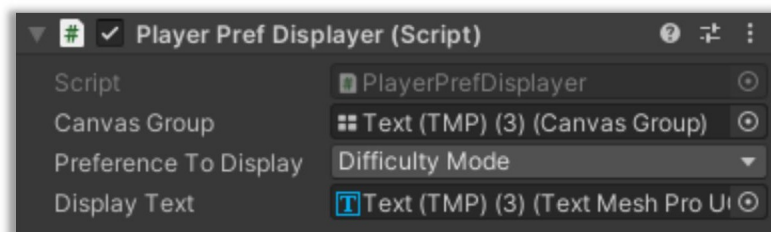
플레이어 환경설정 저장을 위한 열거형을 포함합니다. 여러 데모 스크립트에서 참조합니다.

PreferenceTracker



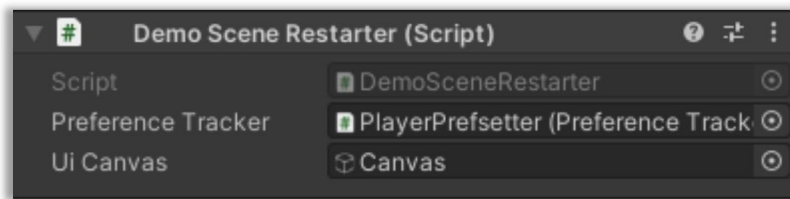
플레이어 환경설정 상호작용을 제어하는 간소화된 설정 관리자입니다.

PlayerPrefDisplayer



텍스트 객체에 설정된 플레이어 기본 설정을 표시하는 데 사용됩니다.

데모 장면 재시작

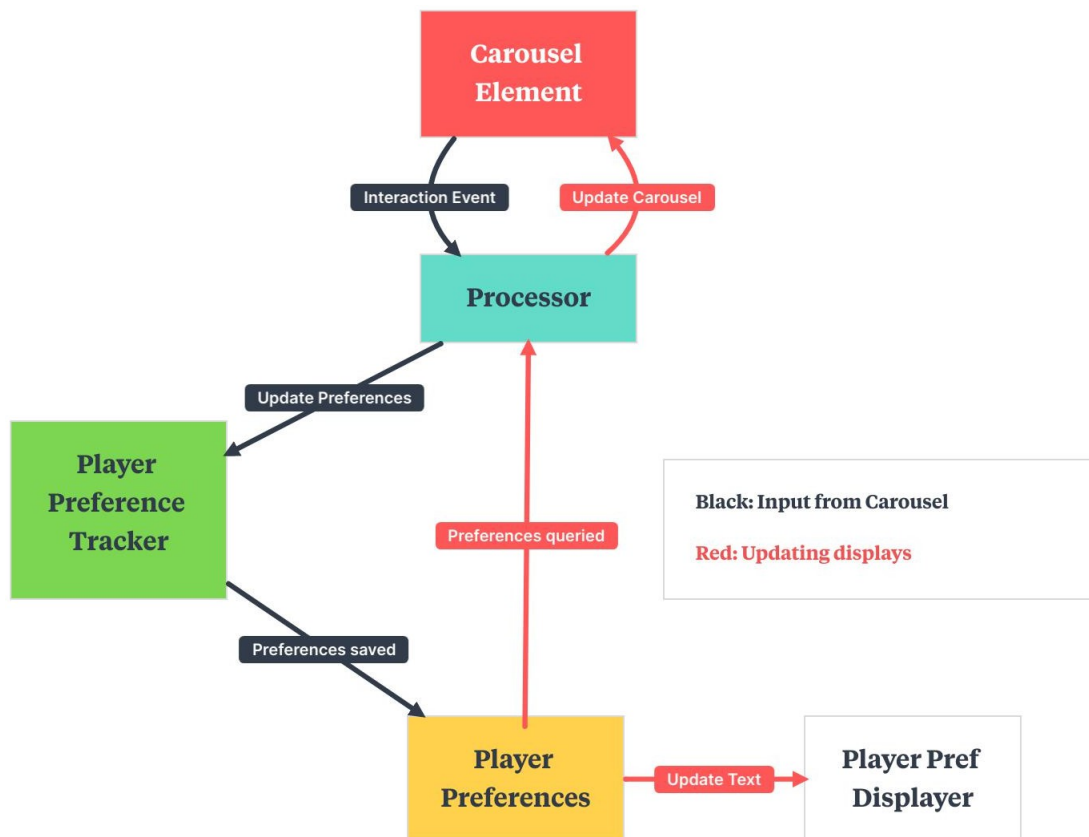


게임 오브젝트를 빠르게 비활성화 및 활성화하는 간단한 스크립트입니다. 씬 표시를 재설정하는 데 사용됩니다.

입력 저장

이 측면에 대한 자세한 내용은 이 사용 설명서의 범위를 벗어나지만 데모에 사용된 플레이어 환경 설정 접근 방식에 대해 간략하게 설명하겠습니다.

데모 구현에서 플레이어 환경설정은 PreferenceTracker 스크립트로 관리되며, 씬의 모든 CarouselUIProcessor에서 참조합니다. 캐러셀 요소와 상호 작용할 때마다 관련 프로세서가 트리거되며, 이 프로세서는 PreferenceTracker 스크립트를 통해 제어하는 설정에 따라 값을 설정합니다. 이 변경 사항은 내장된 플레이어 환경설정 시스템에 저장되어 장면이 언로드되거나 애플리케이션이 종료된 후에도 저장된 값을 유지할 수 있습니다.



데모 장면이 다시 로드되면(또는 프로세서에서 OnEnable()이 호출되면) 캐러셀 프로세서 스크립트는 기본 설정 트래커를 통해 제어하는 설정 값을 조회합니다. 거기에서 내부 데이터와 관련 캐러셀을 적절한 저장된 값으로 설정합니다.

플레이어 환경설정을 활용하는 것이 설정 데이터를 보관하는 가장 쉬운 방법 중 하나이지만, JSON과 같은 다른 대안도 있습니다. 이 경우 JSON 읽기/쓰기를 제어하는 사용자 지정 스크립트가 필요하지만 기본 프로세서 스크립트를 상속하여 캐러셀 시스템에 쉽게 연결할 수 있습니다.

최종 생각

이 에셋은 데모의 범위를 벗어나 더 확장할 수 있습니다. 예를 들어 캐러셀의 옵션에 텍스트 대신 이미지를 사용할 수 있으며, 이를 위해 코드를 실제로 변경할 필요는 없습니다. 또한 활성 옵션의 양쪽에 이전 옵션과 다음 옵션이 동시에 표시되는 "클래식" 캐러셀로 확장할 수도 있습니다. 하지만 이 경우 큰 문제는 아니지만 약간의 코드 수정이 필요합니다.

이 에셋에서 제공하는 리소스를 기반으로 필요에 맞게 확장할 수 있습니다.

팁

1. 프리팹을 사용할 때는 프리팹의 포장을 풀고 옵션을 수동으로 삭제하는 대신 옵션 배열에서 불필요한 레퍼런스를 제거하고 사용하지 않는 게임 오브젝트를 숨기는 것이 좋습니다.
2. UI 프리팹의 시각적 요소는 대부분 표준 Unity 컴포넌트이므로 쉽게 교체할 수 있습니다. 상호 이식성을 위해 데모/프리팹에서 사용하는 레퍼런스는 Unity에 기본으로 제공되는 것으로 제한했습니다. 물론 커스텀 버전은 이러한 제한을 받을 필요가 없습니다.
3. 캐러셀 UI 요소와 캐러셀 프로세서 스크립트가 설계된 방식은 내부적으로 값을 정수로 추적합니다. 이러한 스크립트를 사용할 때는 설정 값을 저장할 때 정수를 계속 사용하는 것이 좋지만, 열거형이나 문자열을 지원하도록 두 스크립트의 일부 코드를 변경하는 것은 특별히 어렵지 않습니다.